UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026

INFORME DE GUÍA PRÁCTICA #1

I. PORTADA

Tema:	Gestión de colección de datos utilizando vectores.
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero B
Alumnos participantes:	Nata Analuisa Alexis Walter Sigcha Arcos Justin Israel
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Desarrollar una aplicación en C++ basada en programación orientada a objetos que permita gestionar la información de los estudiantes de un curso, incluyendo sus datos personales y calificaciones, con el fin de facilitar el registro, consulta y cálculo de promedios académicos.

Específicos:

- Diseñar la clase Estudiante para almacenar los datos personales y las calificaciones de cada alumno utilizando estructuras como vectores.
- Implementar una clase Curso que permita administrar el conjunto de estudiantes, incluyendo operaciones como insertar, buscar, modificar y eliminar registros.
- Desarrollar un menú interactivo por consola que facilite la navegación entre las diferentes funcionalidades del sistema.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4
No presenciales: 0

2.4 Instrucciones

Implemente una aplicación para la gestión de los datos de los estudiantes matriculados en un curso con cupo para hasta 20 estudiantes. Para ello, deberá:

1. Definir la clase Estudiante, con atributos cédula de identidad, nombres, apellidos, fecha de nacimiento y un vector con las notas (máximo de 7 notas, asumiendo que es la cantidad máxima de actividades en que se evaluará a un estudiante).
2. Implementar adecuadamente el constructor de la clase Estudiante y un método buscar(), en la clase que considere oportuno, para que busque por cédula a un estudiante dado.
3. Implemente el siguiente menú de opciones en la aplicación C++ por consola:

=== GESTOR DE PERSONAS ===
1 Estudiantes.
2 Registro de calificaciones.
3 Determinar el promedio de notas de un estudiante.
4 Determinar el promedio de notas del curso.
Teclee su opción (1-4)

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computador, Cuadernos, esferos, lápiz, borrador, Aula virtual e Internet

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

El comportamiento del programa cuando el usuario interactúe con el menú de opciones deberá ser como se describe a continuación.
En la opción 1, al usuario se le mostrará un listado con un autonumérico, para identificar cada registro de cada estudiante, y los datos de cada estudiante previamente
registrado. A partir de ello, el usuario podrá acceder a un submenú que le permita ingresar, modificar y eliminar los datos de un estudiante. Cada vez que un usuario
inserte/modifique/elimine un registro, si es que tiene sentido, se le preguntará si desea realizar la misma acción una vez más. Si la cantidad máxima de estudiantes ya fue
registrada, no se permitirá insertar datos de un nuevo estudiante. Si no hay estudiantes registrados, no se le permitirá intentar eliminar el registro de un estudiante. En todo
caso, para las opciones de modificar y eliminar, el usuario comenzará indicando el valor de autonumérico que identifique al registro de estudiante que se quiere
modificar/eliminar.
En la opción 2, el usuario podrá ingresar los datos de calificaciones de un estudiante. Comenzará introduciendo el número de cédula del estudiante en cuestión. Si el
número de cédula pertenece a uno de los estudiantes registrados, se mostrarán sus datos (nombres y apellidos y edad), y se permitirá ingresar tantos valores de
calificaciones como el usuario considere. Se le mostrará un listado de calificaciones previamente registradas. Se permitirá modificar o eliminar cada una de estas, o insertar
nuevas calificaciones. Si se llega a ingresar la cantidad máxima de calificaciones posible, el sistema notificará que se han ingresado todas las calificaciones posibles y se
da por terminado el proceso de entrada de calificaciones. Si el número de cédula ingresado inicialmente no fuera el de alguno de los estudiantes registrados, se le notificará
oportunamente al usuario y se le dará la opción de ingresar otro número de cédula o regresar al menú principal.
En la opción 3, el usuario comenzará introduciendo el número de cédula del estudiante del que se quiere calcular el promedio de calificaciones. Si el número de cédula
pertenece a uno de los estudiantes ya registrados, se mostrarán sus datos (nombres y apellidos, edad y promedio de calificaciones). Caso contrario, se emitirá un mensaje
de error indicando que no se encontró un estudiante con el número de cédula indicado.
En la opción 4, se mostrará un mensaje indicando el valor del promedio general de calificaciones. Si ningún estudiante tiene calificaciones registradas, de igual manera, se
indicará que No se han registrado calificaciones de estudiantes.
Nota: el conjunto de estudiantes, y el conjunto de notas por estudiante, se almacenará en vectores

2.7 Resultados obtenidos

En el desarrollo de este sistema de gestión de estudiantes se aplican conceptos fundamentales de la programación orientada a objetos, como el uso de clases, encapsulamiento y modularidad, con el fin de organizar y manejar la información de manera eficiente. El programa permite registrar, modificar y consultar datos de estudiantes, así como gestionar sus calificaciones y promedios.

Para el almacenamiento de los datos se utiliza la estructura dinámica vector, la cual forma parte de la biblioteca estándar de C++. Se eligió el uso de vectores porque permiten manejar una cantidad variable de elementos sin necesidad de definir un tamaño fijo desde el inicio, facilitando la inserción y eliminación de estudiantes de forma dinámica. Además, los vectores proporcionan acceso directo a los elementos mediante índices, lo que hace más eficiente la manipulación de la información en comparación con otras estructuras.[1]

De esta manera, el uso de vectores junto con la organización en clases permite construir un sistema más flexible, escalable y fácil de mantener, cumpliendo con los requerimientos del programa.

Estudiante.h

```cpp
#ifndef ESTUDIANTE_H
#define ESTUDIANTE_H

#include <string>
#include <vector>

using namespace std;

class Estudiante {
private:
    string cedula;
    string nombres;
    string apellidos;
    int    dia_nac;
    int    mes_nac;
    int    anio_nac;
    vector<double> notas;  // maximo 7 notas

public:
    // Constructores
    Estudiante();
    Estudiante(const string& cedula, const string& nombres,
               const string& apellidos, int dia, int mes, int anio);

    // Getters
    string getCedula()     const;
    string getNombres()    const;
    string getApellidos()  const;
    int    getDia()        const;
    int    getMes()        const;
    int    getAnio()       const;
    vector<double> getNotas() const;
    int    getCantNotas()  const;
```

```
    // Setters
    void setCedula(const string& cedula);
    void setNombres(const string& nombres);
    void setApellidos(const string& apellidos);
    void setDia(int dia);
    void setMes(int mes);
    void setAnio(int anio);

    // Metodos de notas
    void   agregarNota(double nota);
    void   modificarNota(int indice, double nota);
    void   eliminarNota(int indice);

    // Metodos de calculo
    int    calcularEdad()     const;
    double promedioNotas()    const;
};

#endif
```

Se muestra la declaración de la clase Estudiante con todos sus atributos privados (cedula, nombres, apellidos, dia_nac, mes_nac, anio_nac y el vector<double> notas). Al ser privados, no pueden ser accedidos directamente desde fuera de la clase. Se declaran también los dos constructores, los getters, setters y métodos propios como calcularEdad() y promedioNotas().

Estudiante.cpp

```
#include "Estudiante.h"
#include <ctime>

// Constructores
Estudiante::Estudiante()
    : cedula(""), nombres(""), apellidos(""),
      dia_nac(1), mes_nac(1), anio_nac(2000) {}

Estudiante::Estudiante(const string& cedula, const string& nombres,
                       const string& apellidos, int dia, int mes, int anio)
    : cedula(cedula), nombres(nombres), apellidos(apellidos),
      dia_nac(dia), mes_nac(mes), anio_nac(anio) {}

//  Getters
string Estudiante::getCedula()     const { return cedula; }
string Estudiante::getNombres()    const { return nombres; }
string Estudiante::getApellidos()  const { return apellidos; }
int    Estudiante::getDia()        const { return dia_nac; }
int    Estudiante::getMes()        const { return mes_nac; }
int    Estudiante::getAnio()       const { return anio_nac; }
int    Estudiante::getCantNotas()  const { return (int)notas.size(); }

vector<double> Estudiante::getNotas() const { return notas; }

//  Setters
void Estudiante::setCedula(const string& c)     { cedula    = c; }
void Estudiante::setNombres(const string& n)    { nombres   = n; }
void Estudiante::setApellidos(const string& a)  { apellidos = a; }
void Estudiante::setDia(int d)   { dia_nac  = d; }
void Estudiante::setMes(int m)   { mes_nac  = m; }
void Estudiante::setAnio(int a)  { anio_nac = a; }

//  Métodos de notas
void Estudiante::agregarNota(double nota) {
    notas.push_back(nota);
}
```

```
void Estudiante::modificarNota(int indice, double nota) {
    if (indice >= 0 && indice < (int)notas.size())
        notas[indice] = nota;
}

void Estudiante::eliminarNota(int indice) {
    if (indice >= 0 && indice < (int)notas.size())
        notas.erase(notas.begin() + indice);
}
```

Se implementan los dos constructores: el por defecto inicializa los atributos con valores base, y el parametrizado recibe todos los datos del estudiante. Los métodos agregarNota(), modificarNota() y eliminarNota() manipulan el vector interno de calificaciones usando push_back() y erase() de la STL.

```
// ── Métodos de cálculo ──────────────────────────────────────
int Estudiante::calcularEdad() const {
    time_t t   = time(nullptr);
    tm*    hoy = localtime(&t);
    int anio_h = hoy->tm_year + 1900;
    int mes_h  = hoy->tm_mon  + 1;
    int dia_h  = hoy->tm_mday;

    int edad = anio_h - anio_nac;
    if (mes_h < mes_nac || (mes_h == mes_nac && dia_h < dia_nac))
        edad--;
    return edad;
}

double Estudiante::promedioNotas() const {
    if (notas.empty()) return 0.0;
    double suma = 0.0;
    for (double n : notas) suma += n;
    return suma / notas.size();
}
```

calcularEdad() obtiene la fecha actual del sistema con time() y la compara con la fecha de nacimiento para calcular los años cumplidos exactos. promedioNotas() recorre el vector de notas acumulando la suma y divide entre el número de elementos para obtener el promedio.

Curso.h

```
#ifndef CURSO_H
#define CURSO_H

#include "Estudiante.h"
#include <vector>

using namespace std;

class Curso {
private:
    vector<Estudiante> estudiantes;
    static const int MAX_ESTUDIANTES = 20;
    static const int MAX_NOTAS       = 7;

public:
    Curso();

    // Buscar estudiante por cedula, retorna indice o -1
    int buscar(const string& cedula) const;

    // Getters de estado
    int  getCantEstudiantes() const;
    bool estaLleno()          const;
    bool estaVacio()          const;
    Estudiante& getEstudiante(int indice);

    // CRUD estudiantes
    bool insertar(const Estudiante& e);
    bool modificar(int indice, const string& cedula, const string& nombres,
                   const string& apellidos, int dia, int mes, int anio);
    bool eliminar(int indice);

    // Calificaciones
    bool agregarNota(int indice, double nota);
    bool modificarNota(int indice, int indNota, double nota);
    bool eliminarNota(int indice, int indNota);

    // Promedios
    double promedioCurso() const;
    bool   hayCalificaciones() const;

    // Constantes accesibles
    int getMaxEstudiantes() const;
    int getMaxNotas()       const;
};

#endif
```

En este archivo se define la clase Curso, la cual almacena un conjunto de estudiantes.

Se utiliza un vector para guardar los estudiantes y se establecen límites como:

Máximo 20 estudiantes
Máximo 7 notas por estudiante

También se declaran métodos para:

Insertar, modificar y eliminar estudiantes
Buscar estudiantes por cédula
Manejar calificaciones
Calcular el promedio del curso

Esta clase organiza todo el sistema a nivel general.

Curso.cpp

```cpp
#include "Curso.h"

//  Constructor
Curso::Curso() {}

//  buscar
int Curso::buscar(const string& cedula) const {
    for (int i = 0; i < (int)estudiantes.size(); i++)
        if (estudiantes[i].getCedula() == cedula)
            return i;
    return -1;
}

//  Getters de estado
int  Curso::getCantEstudiantes() const { return (int)estudiantes.size(); }
bool Curso::estaLleno()          const { return (int)estudiantes.size() >= MAX_ESTUDIANTES; }
bool Curso::estaVacio()          const { return estudiantes.empty(); }
int  Curso::getMaxEstudiantes()  const { return MAX_ESTUDIANTES; }
int  Curso::getMaxNotas()        const { return MAX_NOTAS; }

Estudiante& Curso::getEstudiante(int indice) {
    return estudiantes[indice];
}

//  CRUD estudiantes
bool Curso::insertar(const Estudiante& e) {
    if (estaLleno()) return false;
    if (buscar(e.getCedula()) != -1) return false;  // cedula duplicada
    estudiantes.push_back(e);
    return true;
}

bool Curso::modificar(int indice, const string& cedula, const string& nombres,
                      const string& apellidos, int dia, int mes, int anio) {
    if (indice < 0 || indice >= (int)estudiantes.size()) return false;
    // Verificar cedula duplicada (excluir el propio registro)
    int idx = buscar(cedula);
    if (idx != -1 && idx != indice) return false;

    if (!cedula.empty())    estudiantes[indice].setCedula(cedula);
    if (!nombres.empty())   estudiantes[indice].setNombres(nombres);
    if (!apellidos.empty()) estudiantes[indice].setApellidos(apellidos);
    if (dia  > 0) estudiantes[indice].setDia(dia);
    if (mes  > 0) estudiantes[indice].setMes(mes);
    if (anio > 0) estudiantes[indice].setAnio(anio);
    return true;
}

bool Curso::eliminar(int indice) {
    if (indice < 0 || indice >= (int)estudiantes.size()) return false;
    estudiantes.erase(estudiantes.begin() + indice);
    return true;
}

//  Calificaciones
bool Curso::agregarNota(int indice, double nota) {
    if (indice < 0 || i bool Curso::agregarNota(int indice, double nota)
    if (estudiantes[indice].getCantNotas() >= MAX_NOTAS)  return false;
    estudiantes[indice].agregarNota(nota);
    return true;
}

bool Curso::modificarNota(int indice, int indNota, double nota) {
    if (indice < 0 || indice >= (int)estudiantes.size()) return false;
    estudiantes[indice].modificarNota(indNota, nota);
    return true;
}
```

```
bool Curso::eliminarNota(int indice, int indNota) {
    if (indice < 0 || indice >= (int)estudiantes.size()) return false;
    estudiantes[indice].eliminarNota(indNota);
    return true;
}

// Promedios
bool Curso::hayCalificaciones() const {
    for (const Estudiante& e : estudiantes)
        if (e.getCantNotas() > 0) return true;
    return false;
}

double Curso::promedioCurso() const {
    double suma  = 0.0;
    int    total = 0;
    for (const Estudiante& e : estudiantes) {
        for (double n : e.getNotas()) {
            suma += n;
            total++;
        }
    }
    return total > 0 ? suma / total : 0.0;
}
```

Aquí se implementa la lógica de la clase Curso.

Se controla:

Validación de datos (ej: no repetir cédulas)
Límite de estudiantes y notas
Gestión completa de estudiantes y calificaciones

Además, se calcula el promedio general del curso sumando todas las notas de todos los estudiantes.

Este archivo es el encargado de manejar todo el funcionamiento del sistema.

Main

```
//  Utilidades de E/S

void limpiarBuffer() {
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

void pausar() {
    cout << "\nPresione ENTER para continuar...";
    limpiarBuffer();
}

void limpiarPantalla() {
    cout << "\033[2J\033[1;1H";
}

int leerEntero(const string& prompt, int minVal, int maxVal) {
    int valor;
    while (true) {
        cout << prompt;
        if (cin >> valor && valor >= minVal && valor <= maxVal) {
            limpiarBuffer();
            return valor;
        }
        cin.clear();
        limpiarBuffer();
        cout << "   Ingrese un numero entre " << minVal << " y " << maxVal << ".\n";
    }
}
```

```
double leerDouble(const string& prompt, double minVal, double maxVal) {
    double valor;
    while (true) {
        cout << prompt;
        if (cin >> valor && valor >= minVal && valor <= maxVal) {
            limpiarBuffer();
            return valor;
        }
        cin.clear();
        limpiarBuffer();
        cout << "   Ingrese un numero entre " << minVal << " y " << maxVal << ".\n";
    }
}

string leerLinea(const string& prompt) {
    string linea;
    while (true) {
        cout << prompt;
        getline(cin, linea);
        size_t ini = linea.find_first_not_of(" \t");
        if (ini != string::npos) {
            linea = linea.substr(ini);
            size_t fin = linea.find_last_not_of(" \t");
            linea = linea.substr(0, fin + 1);
        }
        if (!linea.empty()) return linea;
        cout << "  [!] El campo no puede estar vacio.\n";
    }
}

bool confirmar(const string& pregunta) {
    char resp;
    cout << pregunta << " (s/n): ";
    cin  >> resp;
    limpiarBuffer();
    return (resp == 's' || resp == 'S');
}
```

Este bloque contiene funciones auxiliares que permiten manejar la interacción con el usuario, como la lectura de datos, validación de entradas, limpieza del buffer y control de la pantalla. Su objetivo es evitar errores al ingresar información y mejorar la experiencia del usuario dentro del sistema.

```
//  Mostrar listado de estudiantes

void mostrarListado(Curso& curso) {
    cout << "\n"
         << left
         << setw(4)  << "#"
         << setw(14) << "Cedula"
         << setw(20) << "Nombres"
         << setw(20) << "Apellidos"
         << setw(12) << "Fec.Nac"
         << setw(5)  << "Edad"
         << "\n"
         << string(75, '-') << "\n";

    for (int i = 0; i < curso.getCantEstudiantes(); i++) {
        Estudiante& e = curso.getEstudiante(i);
        string fec = to_string(e.getDia())  + "/" +
                     to_string(e.getMes())  + "/" +
                     to_string(e.getAnio());
        cout << left
             << setw(4)  << (i + 1)
             << setw(14) << e.getCedula()
             << setw(20) << e.getNombres()
             << setw(20) << e.getApellidos()
             << setw(12) << fec
             << setw(5)  << e.calcularEdad()
             << "\n";
    }
}
```

Este módulo se encarga de presentar la información de los estudiantes registrados en forma de tabla, mostrando datos como cédula, nombres, apellidos, fecha de nacimiento y edad. Permite visualizar la información de manera organizada y clara.

```
//  OPCIÓN 1 — Gestión de estudiantes


void insertarEstudiante(Curso& curso) {
    if (curso.estaLleno()) {
        cout << "\n El curso ya tiene el maximo de "
             << curso.getMaxEstudiantes() << " estudiantes.\n";
        return;
    }
    bool seguir = true;
    while (seguir && !curso.estaLleno()) {
        cout << "\n--- Nuevo estudiante ---\n";
        string ced = leerLinea("  Cedula       : ");
        string nom = leerLinea("  Nombres      : ");
        string ape = leerLinea("  Apellidos    : ");
        int d = leerEntero("  Dia nac.     : ", 1, 31);
        int m = leerEntero("  Mes nac.     : ", 1, 12);
        int a = leerEntero("  Anio nac.    : ", 1900, 2100);

        Estudiante nuevo(ced, nom, ape, d, m, a);
        if (curso.insertar(nuevo))
            cout << "   Estudiante registrado.\n";
        else
            cout << "  Ya existe un estudiante con esa cedula.\n";

        if (!curso.estaLleno())
            seguir = confirmar("  ¿Ingresar otro estudiante?");
        else {
            cout << "   Se alcanzo el maximo de estudiantes.\n";
            seguir = false;
        }
    }
}
void modificarEstudiante(Curso& curso) {
    if (curso.estaVacio()) {
        cout << "\n No hay estudiantes registrados.\n";
        return;
    }
    bool seguir = true;
    while (seguir) {
        mostrarListado(curso);
        int num = leerEntero("\n  Numero a modificar (0 = cancelar): ",
                             0, curso.getCantEstudiantes());
        if (num == 0) return;

        Estudiante& e = curso.getEstudiante(num - 1);
        cout << "\n  (ENTER para conservar valor actual)\n\n";

        cout << "  Cedula [" << e.getCedula() << "]: ";
        string tmp; getline(cin, tmp);
        string ced = tmp.empty() ? "" : tmp;

        cout << "  Nombres [" << e.getNombres() << "]: ";
        getline(cin, tmp);
        string nom = tmp;

        cout << "  Apellidos [" << e.getApellidos() << "]: ";
        getline(cin, tmp);
        string ape = tmp;

        cout << "  Dia [" << e.getDia() << "]: ";
        getline(cin, tmp);
        int d = tmp.empty() ? 0 : stoi(tmp);

        cout << "  Mes [" << e.getMes() << "]: ";
        getline(cin, tmp);
        int m = tmp.empty() ? 0 : stoi(tmp);

        cout << "  Anio [" << e.getAnio() << "]: ";
        getline(cin, tmp);
        int a = tmp.empty() ? 0 : stoi(tmp);
```

```
void eliminarEstudiante(Curso& curso) {
    if (curso.estaVacio()) {
        cout << "\nNo hay estudiantes para eliminar.\n";
        return;
    }
    bool seguir = true;
    while (seguir && !curso.estaVacio()) {
        mostrarListado(curso);
        int num = leerEntero("\n  Numero a eliminar (0 = cancelar): ",
                             0, curso.getCantEstudiantes());
        if (num == 0) return;

        Estudiante& e = curso.getEstudiante(num - 1);
        cout << "  Eliminar: " << e.getNombres() << " " << e.getApellidos() << "\n";
        if (confirmar("  ¿Confirma?")) {
            curso.eliminar(num - 1);
            cout << "   Estudiante eliminado.\n";
        } else {
            cout << "  Cancelado.\n";
        }
        if (!curso.estaVacio())
            seguir = confirmar("  ¿Eliminar otro estudiante?");
    }
}
void menuEstudiantes(Curso& curso) {
    int op;
    do {
        limpiarPantalla();
        cout << "\n=== GESTION DE ESTUDIANTES ===\n";
        cout << "Registrados: " << curso.getCantEstudiantes()
             << " / " << curso.getMaxEstudiantes() << "\n";

        if (!curso.estaVacio())
            mostrarListado(curso);
        else
            cout << "\n  (Sin estudiantes registrados)\n";

        cout << "\n  1. Ingresar\n"
             << "  2. Modificar\n"
             << "  3. Eliminar\n"
             << "  0. Volver\n";
        op = leerEntero("\n  Opcion (0-3): ", 0, 3);

        switch (op) {
            case 1: insertarEstudiante(curso); pausar(); break;
            case 2: modificarEstudiante(curso); pausar(); break;
            case 3: eliminarEstudiante(curso);  pausar(); break;
        }
    } while (op != 0);
}
```

Este bloque incluye las funciones para insertar, modificar y eliminar estudiantes, así como un submenú que agrupa estas opciones. Se encarga de la interacción con el usuario para gestionar los datos, mientras que la lógica real se delega a la clase Curso.

```
//  OPCIÓN 2 — Registro de calificaciones

void menuCalificaciones(Curso& curso) {
    if (curso.estaVacio()) {
        cout << "\n[!] No hay estudiantes registrados.\n";
        pausar();
        return;
    }

    bool buscarOtro = true;
    while (buscarOtro) {
        limpiarPantalla();
        cout << "\n=== REGISTRO DE CALIFICACIONES ===\n";
        string ced = leerLinea("  Cedula del estudiante: ");
        int idx = curso.buscar(ced);

        if (idx == -1) {
            cout << "   No se encontro estudiante con cedula: " << ced << "\n";
            buscarOtro = confirmar("  ¿Ingresar otra cedula?");
            continue;
        }

        Estudiante& e = curso.getEstudiante(idx);
        bool gestionando = true;

        while (gestionando) {
            limpiarPantalla();
            cout << "\n=== CALIFICACIONES ===\n";
            cout << "  Estudiante : " << e.getNombres() << " " << e.getApellidos() << "\n";
            cout << "  Edad       : " << e.calcularEdad() << " anios\n";
            cout << "  Notas (" << e.getCantNotas() << "/" << curso.getMaxNotas() << "):\n";

            if (e.getCantNotas() == 0) {
                cout << "    (Sin calificaciones)\n";
            } else {
                vector<double> notas = e.getNotas();
                for (int i = 0; i < (int)notas.size(); i++)
                    cout << "    " << (i+1) << ". "
                         << fixed << setprecision(2) << notas[i] << "\n";
            }

            if (e.getCantNotas() >= curso.getMaxNotas()) {
                cout << "\n   Se ingresaron todas las calificaciones posibles ("
                     << curso.getMaxNotas() << "/" << curso.getMaxNotas() << ").\n";
                pausar();
                gestionando = false;
                buscarOtro  = false;
                break;
            }

            cout << "\n  1. Insertar calificacion\n";
            if (e.getCantNotas() > 0)
                cout << "  2. Modificar calificacion\n"
                     << "  3. Eliminar calificacion\n";
            cout << "  0. Volver\n";

            int maxOp = (e.getCantNotas() > 0) ? 3 : 1;
            int op = leerEntero("\n  Opcion: ", 0, maxOp);

            switch (op) {
                case 0:
                    gestionando = false;
                    buscarOtro  = false;
                    break;
                case 1: {
                    double nota = leerDouble("  Calificacion : ", 0.0, 10.0);
                    curso.agregarNota(idx, nota);
                    cout << "   Calificacion registrada.\n";
                    pausar();
```

```
                    break;
                }
                case 2: {
                    int num = leerEntero("  Numero de nota a modificar: ", 1, e.getCantNotas());
                    double nota = leerDouble("  Nueva calificacion (0.0 - 10.0): ", 0.0, 10.0);
                    curso.modificarNota(idx, num - 1, nota);
                    cout << "   Calificacion actualizada.\n";
                    pausar();
                    break;
                }
                case 3: {
                    int num = leerEntero("  Numero de nota a eliminar: ", 1, e.getCantNotas());
                    curso.eliminarNota(idx, num - 1);
                    cout << "   Calificacion eliminada.\n";
                    pausar();
                    break;
                }
            }
        }
    }
}
```

Este módulo permite administrar las notas de los estudiantes, incluyendo la inserción, modificación y eliminación de calificaciones. Además, muestra las notas registradas y controla que se mantengan dentro de los límites establecidos.

```
//  OPCIÓN 3 — Promedio de un estudiante
void promedioEstudiante(Curso& curso) {
    if (curso.estaVacio()) {
        cout << "\n No hay estudiantes registrados.\n";
        pausar();
        return;
    }
    limpiarPantalla();
    cout << "\n=== PROMEDIO DE UN ESTUDIANTE ===\n";
    string ced = leerLinea("  Cedula del estudiante: ");
    int idx = curso.buscar(ced);

    if (idx == -1) {
        cout << "   No se encontro un estudiante con cedula: " << ced << "\n";
    } else {
        Estudiante& e = curso.getEstudiante(idx);
        cout << "\n  Nombres  : " << e.getNombres() << " " << e.getApellidos() << "\n";
        cout << "  Edad     : " << e.calcularEdad() << " anios\n";
        if (e.getCantNotas() == 0) {
            cout << "  Promedio : (sin calificaciones registradas)\n";
        } else {
            cout << "  Notas    : ";
            vector<double> notas = e.getNotas();
            for (int i = 0; i < (int)notas.size(); i++) {
                cout << fixed << setprecision(2) << notas[i];
                if (i < (int)notas.size()-1) cout << ", ";
            }
            cout << "\n";
            cout << "  Promedio : " << fixed << setprecision(2)
                 << e.promedioNotas() << "\n";
        }
    }
    pausar();
}
```

Esta función permite calcular y mostrar el promedio de un estudiante específico, buscando por su cédula y presentando tanto sus calificaciones como el resultado final de su rendimiento académico.

```
//  OPCIÓN 4 — Promedio general del curso

void promedioCurso(Curso& curso) {
    limpiarPantalla();
    cout << "\n=== PROMEDIO GENERAL DEL CURSO ===\n";
    if (!curso.hayCalificaciones()) {
        cout << "  No se han registrado calificaciones de estudiantes.\n";
    } else {
        cout << "  Promedio general: "
             << fixed << setprecision(2) << curso.promedioCurso() << "\n";
    }
    pausar();
}
```

Este bloque calcula el promedio general del curso tomando en cuenta las calificaciones de todos los estudiantes, permitiendo obtener una visión global del rendimiento académico.

```
int main() {
    Curso curso;
    int opcion;

    do {
        limpiarPantalla();
        cout << "\n";

        cout << "       === GESTOR DE PERSONAS ===   \n";

        cout << "    1  Estudiantes                  \n";
        cout << "    2  Registro de calificaciones   \n";
        cout << "    3  Promedio de un estudiante    \n";
        cout << "    4  Promedio del curso           \n";
        cout << "    0  Salir                        \n";

        opcion = leerEntero("\n  Teclee su opcion (0-4): ", 0, 4)

        switch (opcion) {
            case 1: menuEstudiantes(curso);     break;
            case 2: menuCalificaciones(curso); break;
            case 3: promedioEstudiante(curso); break;
            case 4: promedioCurso(curso);       break;
            case 0: cout << "\n  Hasta luego.\n\n"; break;
        }
    } while (opcion != 0);

    return 0;
}
```

El main actúa como el controlador principal del sistema, mostrando el menú, recibiendo las opciones del usuario y llamando a las funciones correspondientes. Permite la navegación entre los diferentes módulos del programa y mantiene la ejecución hasta que el usuario decide salir.

2.8 Habilidades blandas empleadas en la práctica

☒ Trabajo en equipo
☒ Responsabilidad

2.9 Conclusiones

El desarrollo del sistema permitió aplicar de manera práctica los conceptos de programación orientada a objetos, como el uso de clases, métodos y encapsulamiento, logrando una mejor organización del código. Además, se comprendió la importancia de separar la lógica del programa en diferentes módulos, facilitando su mantenimiento y comprensión.

El uso de la estructura vector resultó adecuado para el manejo dinámico de los estudiantes, ya que permitió agregar, modificar y eliminar registros sin necesidad de definir un tamaño fijo. Esto hizo que el sistema sea más flexible y adaptable a diferentes cantidades de datos.

Finalmente, se logró implementar un sistema funcional capaz de gestionar información de estudiantes y sus calificaciones, cumpliendo con los requerimientos planteados y fortaleciendo las habilidades en el manejo de estructuras de datos en C++.

2.10 Recomendaciones

Se recomienda seguir mejorando el sistema implementando una interfaz gráfica, ya que esto permitiría una interacción más amigable para el usuario y facilitaría su uso.

También sería útil agregar validaciones más robustas en el ingreso de datos, para evitar errores como entradas incorrectas o inconsistentes en la información registrada.

2.11 Referencias bibliográficas

[1] "Vectores en C++ STL - GeeksforGeeks." Accessed: Mar. 28, 2026. [Online]. Available: https://www-geeksforgeeks-org.translate.goog/cpp/vector-in-cpp-stl/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true

2.12 Anexos

GitHub

https://github.com/Alexis112008/Ape-N2.git

Menú principal

```
      === GESTOR DE PERSONAS ===
   1  Estudiantes
   2  Registro de calificaciones
   3  Promedio de un estudiante
   4  Promedio del curso
   0  Salir

 Teclee su opcion (0-4): |
```

Opción1:

Ingresar

```
=== GESTION DE ESTUDIANTES ===
Registrados: 0 / 20

  (Sin estudiantes registrados)

  1. Ingresar
  2. Modificar
  3. Eliminar
  0. Volver

  Opcion (0-3): 1

--- Nuevo estudiante ---
  Cedula       : 1850630938
  Nombres      : jostin
  Apellidos    : sighca
  Dia nac.     : 04
  Mes nac.     : 04
  Anio nac.    : 2006
   Estudiante registrado.
  ┐Ingresar otro estudiante? (s/n): |
```

Modificar

```
 Opcion (0-3): 2

#   Cedula        Nombres            Apellidos          Fec.Nac     Edad
-------------------------------------------------------------------------
1   1850630938    jostin             sighca             4/4/2006    19

 Numero a modificar (0 = cancelar): 1

 (ENTER para conservar valor actual)

 Cedula [1850630938]:
 Nombres [jostin]:
 Apellidos [sighca]:
 Dia [4]:
 Mes [4]:
 Anio [2006]: 2005
  Datos actualizados.
```

Opción2: Registro de Calificaciones

```
=== CALIFICACIONES ===
  Estudiante : jostin sighca
  Edad       : 20 anios
  Notas (2/7):
    1. 5.00
    2. 8.00

  1. Insertar calificacion
  2. Modificar calificacion
  3. Eliminar calificacion
  0. Volver

  Opcion: |
```

Opción3: Promedio de un estudiante

```
=== PROMEDIO DE UN ESTUDIANTE ===
  Cedula del estudiante: 1850630938

  Nombres  : jostin sighca
  Edad     : 20 anios
  Notas    : 5.00, 8.00
  Promedio : 6.50
```

Opción4: Promedio del curso

```
=== PROMEDIO GENERAL DEL CURSO ===
  Promedio general: 6.50
```

Opción 0: Salir

```
 Teclee su opcion (0-4): 0

 Hasta luego.
```